==========

What is Fullstack

==========

Fullstack Development = Frontend Development + Backend Development

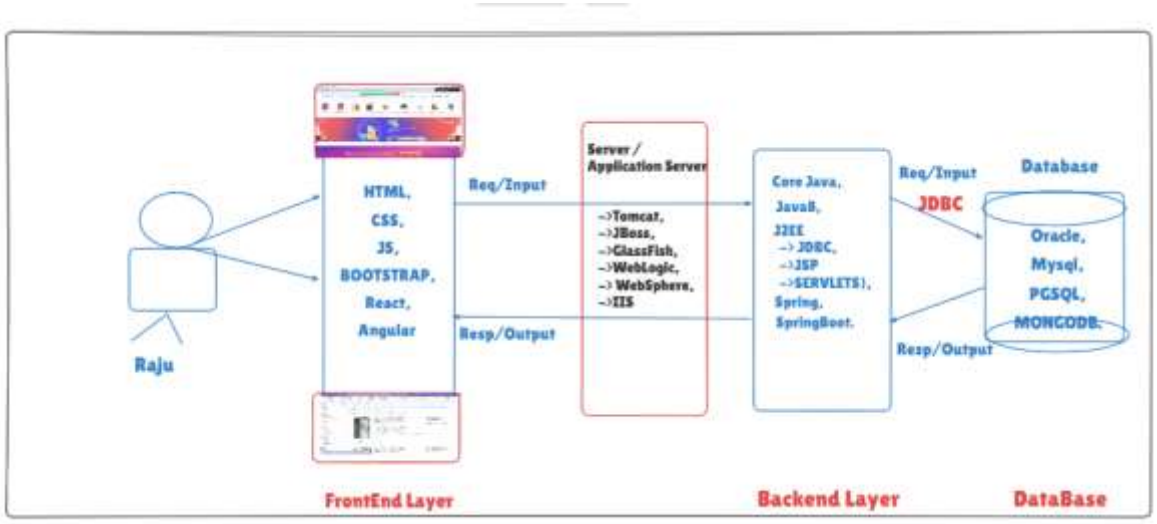
Frontend : User interface

-> Clients / Users will interact with s/w application using frontend

Backend : The hidden part of our application which contains business logic

-> When user perform some operation on frontend then backend logic will execute to handle that operation

-> The programmer who can develop frontend and backend of application is called as fullstack developer



==========

Frontend Technologies

==========

HTML & CSS

Java script

Bootstap

Angular

React

==========

Backend Technologies

==========

Java

Python

PHP

.Net

Node JS

==========

Servers

==========

Tomcat

JBoss

Glassfish

Oracle Weblogic

IBM WebSphere

IIS

==========

Databases

==========

Oracle

MySQL

SQL Server

Postgres

Mongo DB

Casandra

Hbase

Hive

========

Tools

=========

Git Hub : For Code Integration

JIRA : Project Management / Bug Tracking / Work Assignment

SonarQube : For Code Quality Checking

JUnit : For Unit Testing

JMETER : For Performance Testing

JENKINS : For Deployment (Automated Deployment)

==========

Cloud

===========

Amazon ---> AWS

Microsoft ---> Azure

Google -----> GCP

Oracle Cloud

IBM Cloud

VM Ware Cloud

Alibaba CLoud etc.....

=====================================

Roles & Responsibilities of Fullstack Developer

=====================================

1) Understand Requirements of Project

2) Analyze requirements

3) Design / Planning

4) Database Design

5) Development (Backend development)

6) Unit Testing

7) Code Review

8) Code Integration (Git Hub)

9) Frontend Development

10) Frontend + Backend Integration

11) Deployment

12) Support / Maintenence

==========

What is Java

==========

-> Java is a programming language

-> Java language developed by Sun Microsystem in 1991 (OAK)

-> James Gosling is the lead for the team who developed Java Language

-> The first version of java came into market in 1995

Note: Oracle Corporation acquired Sun Microsystem

-> Now java is under license of Oracle corporation

-> Java is a free software & open source

==========

Java is divided into 3 parts

==========

1) J2SE

2) J2EE

3) J2ME

J2SE / JSE ---> JAVA STANDARD EDITION

-> STAND-ALONE APPS

-> RUNS ONLY IN ONE MACHINE or ONLY ONE PERSON CAN ACCESS AT TIME

EX: CALC, GAMES, NOTEPAD ETC.....

J2EE / JEE ---> JAVA ENTERPRISE EDITION

-> web applications

-> Everybody can access web applications using internet

ex: gmail, youtube, facebook, naukri, irctc etc.....

J2ME / JME ---> JAVA MICRO / MOBILE EDITION

-> Mobile apps

Ex: whatsapp, messgender, phonepay, gpay etc.....

==========

What we can do using Java

==========

1) Stand-alone applications

2) Web applications

3) Mobile Applications

==========

Java Features

==========

1) Simple : The complex topics of C & C++ are eliminated in Java[OPM]

Ex: Operators overloading, pointers, memory mgmt etc...

2) Platform Independent

-> Java programs can be executed on any machine

-> JVM made java as platform independent

-> JVM stands for Java Virtual Machine

-> JVM is responsible to run/execute java programs

3) Robust (Strong)

-> Automatic Memory Management

-> Exception Handling

4) OOPS (Object Oriented Programming System)

-> Everything will be represented in objects format

-> Code Re-Usability

5) Secure

6) Distributed

7) Portable

8) Dynamic

Java Slogan : WORA (Write Once Run Anywhere)

==========

Environment Setup

==========

1) Download and Install Java Software

- JDK (Java Development Kit)

- JRE (Java Runtime Environment)

Q) What is the difference between JDK, JRE & JVM ?

- JDK contains set of tools to develop java programs

- JRE providing a platform to run our java programs

- JVM will take care of program execution

2) Set Path for Java

Path = C:\Program Files\Java\jdk1.8.0_202\bin

-> Go To Environment Variables

-> Go To System Environment Variables

-> Edit Path

-> Add JDK BIN path

==========

Types of Java Programs Development

==========

1)TEXT EDITORS

2) IDE

-> We can write java programs in any text editor

- Note Pad
- Note Pad++
- Edit Plus

-> In companies we will use IDE to develop java programs/projects

- Integrated Development Environment
  - Eclipse
  - MyEclipse
  - Netbeans
  - STS (Spring Tool Suite)
  - IntelliJ

==========

Java Program Structure

==========

package statements

import statements

class declaration

variables

methods

----------Demo.java---------

```
class hello {
 public static void main(String... args) {
     System.out.println("Welcome To  IT...!!");
    System.out.println("Welcome to Java");
 }
}
```

Compilation : javac Demo.java

Execution: java Demo

```
class Demo {
   public static void main (String... args){
                        System.out.println("Hello World");
                   System.out.println("Welcome to Java");
   }
}
```

-> javac means java compiler which is used to compile java programs

-> java compiler is called as translator

==========

Translators[ICA]

==========

-> It is used to convert from one format to another format

-> 3 types of translators available

1) Interpreter LINE BY LINE

2) Compiler ALL LINE AT A TIME

3) Assembler[AM] AL TO ML

-> Interpreter will convert the program line by line (performance is slow)

-> Compiler will convert all the lines of program at a time (performance is fast)

-> Assembler is used to convert assembler programming languages into machine language (AM)

 Overall Flow of Translator

Source Program

↓

Translator

↓

Machine Code

↓

Execution

Translator

|

|-- Interpreter

|-- Compiler

|-- Assembler

Source Code

↓

Read 1 Line

↓

Translate LineSource Code

↓

Read 1 Line

↓

Translate Line

1Interpreter – LINE BY LINE 1

Source Code

↓

Check All Lines

↓

Translate Entire Program

↓

Create Machine Code

↓

Execute Program

02]Compiler – ALL LINES AT A TIME

Assembly Language (AL)

↓

Assembler

↓

Machine Language (ML)

↓

Execution

03]Assembler – AL → ML 1

Translator	Input	Conversion Style	Speed
Interpreter	High-level Lang	Line by Line	Slow
Compiler	High-level Lang	All at Once	Fast
Assembler	Assembly Lang	AL → ML	Fast

========

JVM

========

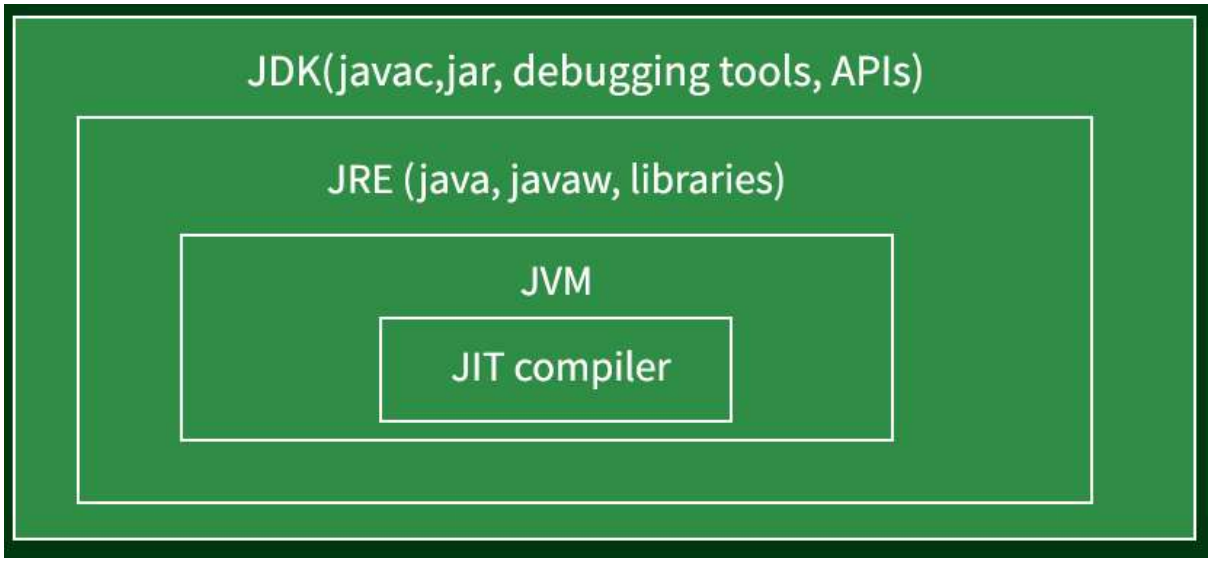
-> JVM stands for Java Virtual Machine (We can't see with our eyes)

-> JVM will be part of JRE

-> JVM is responsible for executing java programs

-> JVM will allocate memory required for program execution & de-allocate memory when it is not used

-> JVM will convert byte code into machine understandable format[B->MC]



COMPILER

==========

SRC ==> BYTE CODE

DE-COMPILER

==========

BYTE CODE ==> SRC

JVM

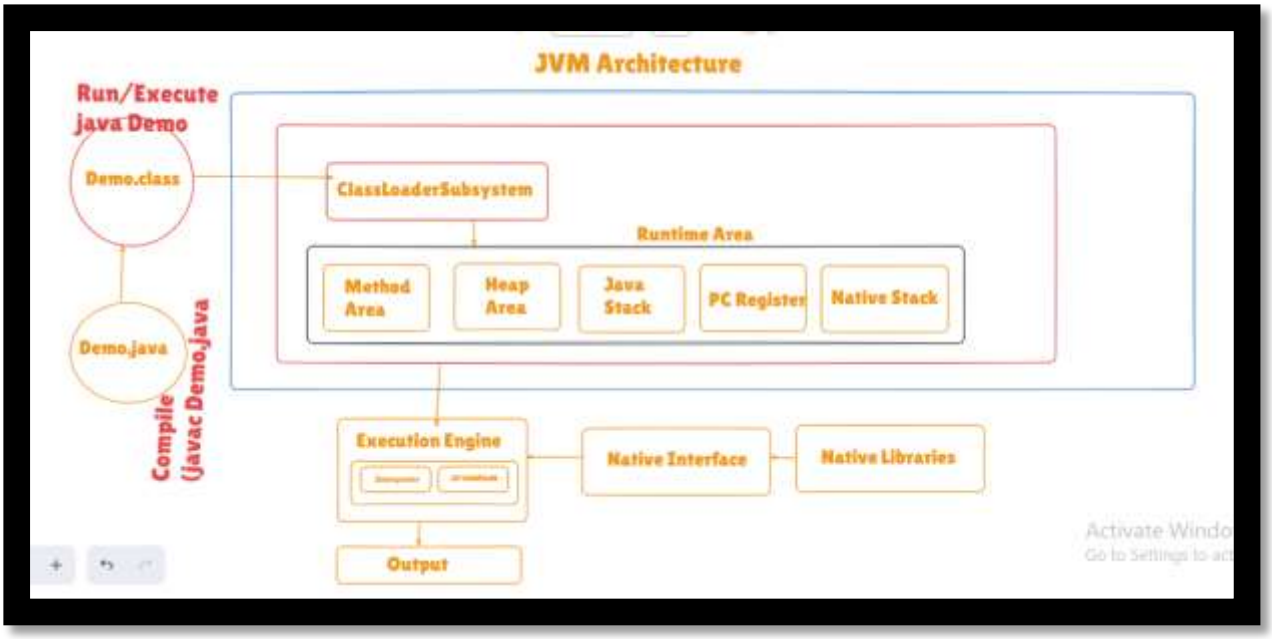
==========

BYTE ==> MACHINE CODE

================

JVM Architecture

===============



1) Classloader subsystem : It will load .class file into JVM

2) Method Area : Class code will be stored here

3) Heap area : Objects will be stored into heap area

4) Java Stack : Method execution information will be stored here

5) PC Register : It will maintain next line information to execute

6) Native Stack : It will maintain non-java code execution information

7) Execution Engine (Interpreter + JIT) : It is responsible to execute the program and provide output/result

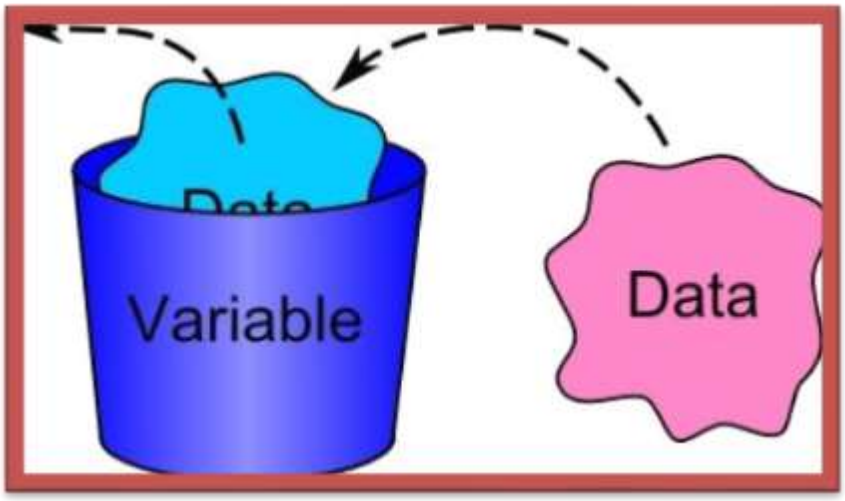
8) Native Interface : It will load native libraries into jvm

9) Native Libraries : Non-java libraries which are required for native code execution

=======

variables

========



-> variables are used to store the data

name - thiru

age - 30

gender - m

isStudent - false

mysalary - 400.56

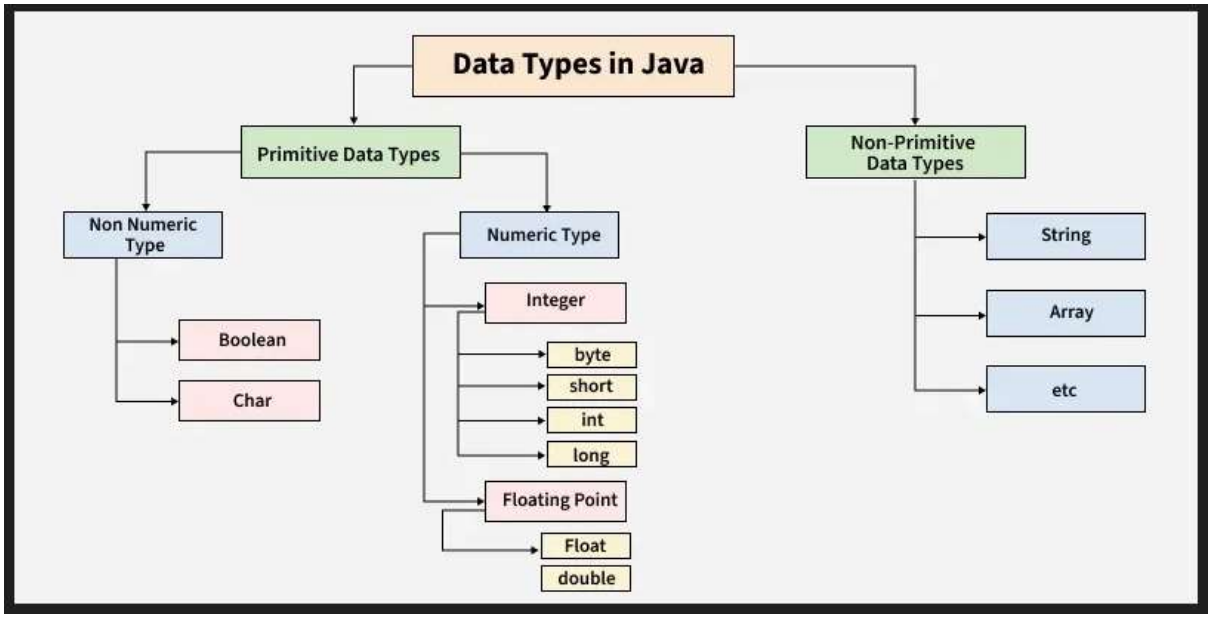
-> We need to specify type of the variable to store the data

-> To specify type of data we will use 'data types'

=========

data types

=========



-> Data types are used to specify type of the data

-> Data types are divided into 2 categories

1) Primitive / Pre-Defined Data Types

1) Integral

- byte
- short
- int
- long

2) Decimal/Floating point

- float
- double

3) Character

- char

4) Boolean

- boolean

2) Non-Primitive / Referenced Data Types

- Arrays
- Strings

- Classes

==> int[] array = new int[5]; // 'array' is a reference to the memory location where the array is stored

==> String myString = "Hello, World!"; // 'myString' is a reference to the memory location where the string is stored

==> MyClass myObject = new MyClass(); // 'myObject' is a reference to the memory location where the object is stored

==> int x = 10; // 'x' directly stores the value 10

==========

Integral data types

==========

--> Integral data types are used to store numbers without decimal points

--> We can store both positive and negative numbers using integral data types

Ex:

age = 30

phno = 66868686868

studentscnt = 40

balance = - 3000

-> We have 4 data types in this category

-> For These 4 data types memory & range is different

1) byte ----> default value is 0 ----> 1 byte
2) short ----> default value is 0 ----> 2 bytes
3) int ----> default value is 0 ----> 4 bytes
4) long ----> default value is 0l ----> 8 bytes

==========

Decimal data types

==========

-> Decimal data types are used to store numbers with decimal values

-> We can store both postive and negative values

Ex:

petrol price = 110.567979

stockPrice = 334.3279797979797979

percentage = 9.8

weight = 55.6

height = 5.6

length = 10.2

-> In this category we have 2 data types

1) float ----> 4 bytes ---> upto 6 decimal points

2) double -----> 8 bytes --> upto 15 decimal points

==========

character data type

==========

-> Character data type is used to store single character

-> Any single character (alphbet / digit / special character) we can store using 'char' data type

-> char datatype will occupy 2 bytes of memory

-> When we are storing data into 'char' data type single quote is mandatory

-> default value is 'u0002'

gender = 'm'

rank = '1'

Note: In C language 'char' will take only 1 byte where as in java 'char' will take 2 bytes

==========

boolean data type

==========

-> It is used to store true or false values only

-> It will occupy 1 bit memory

Note: 8 bits = 1 byte

-> default value for boolean is false

Ex:

isPass;

isFail

isMarried

isOdd

isEven

==========

Variables

==========

-> Variables are used to store the data / value

-> To store the data into variable we need to specify data type

-> To store data into variables we need to perform 2 steps

1) Variable Declaration (defining variable with data type)

Ex: byte age ;

2) Variable Intialization (storing value into variable)

Ex: age = abc;

-> We can complete declaration and intialization in single line

byte age = 20;

========== Variables Program ==========

```
class var {
                    public static void main (String... args)
                    {
                            int age = 20;
                            System.out.println(age);

                            float a = 25.01f;
                            System.out.println(a);

                            double price = 120.87;
                            System.out.println(price);

                            char gender = 'm';
                            System.out.println(gender);

                            boolean pass = true;
                            System.out.println(pass);

    }
}
```

1) Identifiers

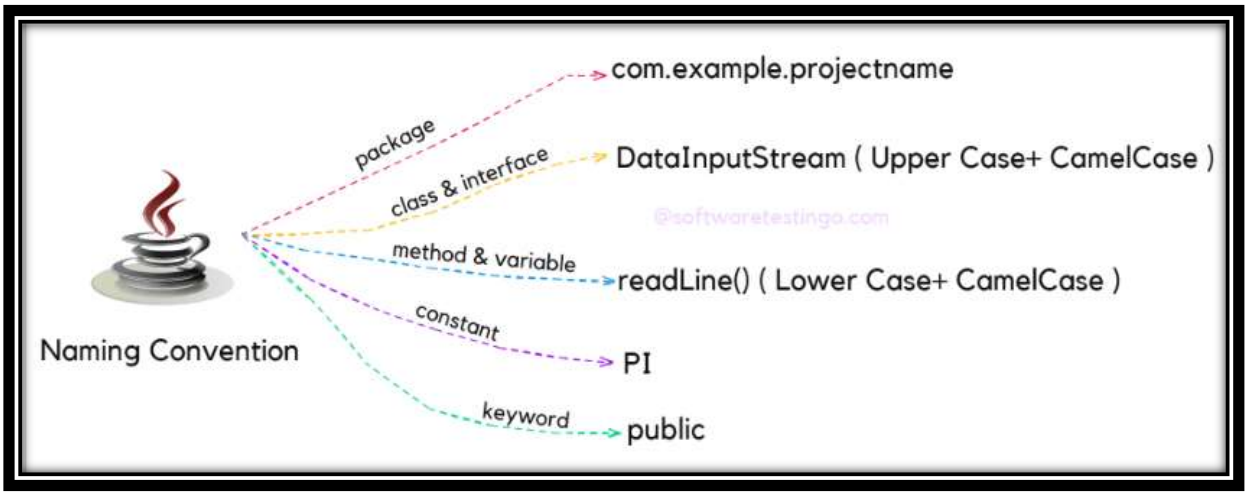
2) Keywords

3) Java Naming Conventions

==========

Identifiers or Naming Conventions

==========



-> All java components requires a name

-> For variables, classes and methods we need a name

```
int age ;

class Hello {

        // code

}


main ( ) {

        //logic

}
```

-> The **name** which we are using for **packages**, **variables**, **classes** & **methods** is called as identifier

-> We can use any name for identifiers but we need to follow below rules to work with identifiers

Rule-1 : Java will allow only below charaters for identifiers

1) a - z

2) A - Z

3) 0 to 9

4) $ (dollar)

5) _ (underscore)

Ex:

name -----> valid

name@ -----> invalid

age# ------> invalid

Rule-2 : Identifier should not start with digit (first character shouldn't be digit)

1age --------> invalid

age2 ------> valid

name3 -----> valid

_name -----> valid

$name ------> valid

@name ------> invalid

$_amt --------> valid

_1bill -----------> valid

Rule-3 : Java **reserved words** shouldn't be used as identifier (**53 reserved words**)

int byte = 20; -------> invalid bcz byte is a reserved word

byte for = 25; -------> invalid bcz for is a reserved word

int try = 30; ------> invalid bcz try is a reserved word

long phno = 797979799 -----> valid

Rule-4 : Spaces are not allowed in identifiers

int mobile bill = 400; // invalid

int mobile_bill = 400 ; // valid

Rule-5 : Java is case sensitive language 'name' & 'NAME' both are not same

==========

Java Naming Conventions (Java Coding Standards)

==========

-> Java language followed some standards/conventions for pre-defined packages, classes and methods....

-> Java language suggested java programmers also to follow same standards / conventions

-> Following these standards/conventions is not mandatory but highly recommended.

==========

Naming Convention For **Class Name**

==========

-> A class name can contain any no.of words without spaces

-> Recommended to write every word first character as uppercase in class name

Examples:

```
class Hello  { }
```

```
class HelloWorld {

}
```

```
class UserManagementService{

}
```

```
class WelcomeRestController {

}
```

Note: Class Names & Interface Names conventions are same.

==========

Variables Naming Convention

==========

-> Variable name can have any no.of words without spaces

-> Recommended to start variable name with lowercase letter

-> If variable name contains multiple words then recommended to write firstword all characters in lowercase and from second word onwards every word first character in Uppercase

Examples:

```
int age ;

int  userAge;

long creditCardNumber ;
```

==========

Method Naming Convention

==========

> Method name can have any no.of words without spaces

-> Recommended to start method name with lowercase letter

-> If method name contains multiple words then recommended to write firstword all characters in lowercase and from second word onwards every word first character in Uppercase

```
main ( )  {

}


save ( ) {

}

saveUser( ) {
```

}

getWelcomeMsg ( ) {

}

Note: Variables & Methods naming conventions are same. But methods will have parenthesis (()) variables will not have parenthesis.

==========

Naming Conventions for Constants

==========

-> Constant means fixed value (value will not change, it is fixed)

-> Recommended to write constant variable all characters in uppercase

-> If constant variable contains multiple words recommended to use _ (underscore) with all uppercase characters

```
final int MIN_AGE = 21;
final int MAX_AGE = 60 ;
int PI = 3.14;
```

==========

Naming Conventions for Packages

==========

-> Package name can have any no.of characters & any of words

-> Recommended to use only lowercase letters in package names

-> If package name conatins multiple words then we will use . (dot) to seperate words

Eamples:

java.lang

java.io

java.util

in.thiru7

com.oracle

com.ibm

==========

Chaper-1

==========

1) What is Java

2) Java Features

3) Java Environment Setup

4) JDK vs JRE vs JVM

5) Java Programs Execution Flow

6) Java Programs Development (Compilation & Execution)

7) Variables

8) Data Types

9) Identifiers

10) Reserved Words (53)

11) Java Coding Standards (Naming Conventions)

12) Java Comments